

- Establish a common vocabulary for architectural concepts so everyone speaks the same language.
- Set aside time to explore design options before starting.
- Treat the architecture haiku as a living document.
- Use the architecture haiku as an outline or executive summary for a longer architecture description.
- The architecture haiku is not a replacement for other design artifacts.

Example

Here's a partial architecture haiku for Project Lionheart. There is also a template available on my website to help get you started.⁶

Project Lionheart is a publicly available web application that will help the Springfield Office of Management and Budget manage the city's requests for proposals (RFPs) and local businesses to find RFPs of interest.

Business Goals

- Reduce procurement costs by 30%
- Improve city engagement with local businesses
- Cut the time required to publish a new RFP in half

Top Quality Attributes

Security > Availability > Performance

Architecture Patterns Used

Service-Oriented Architecture (SOA), Layered web application, REST APIs for web services

Key Decisions and Rationale

- Node.js for web app—team has experience
 - MySQL database—free, open source
 - Apache Solr—free, open source
 - SOA with REST—decouple components, team interested in experimenting with emerging tech trends
 - Java for web services—open source, low-risk, great tool support
-

6. <http://neverletdown.net/2015/03/architecture-haiku.html>

Activity 22

Context Diagram

A *context diagram* helps stakeholders understand where the software system fits in the world. Context diagrams show the people and systems that interact with the software system you are responsible for developing.

Benefits

- Provide a high-level overview of the systems and stakeholder groups the system directly interacts with or relies on.
- Make the boundary between the system you're building and the outside world obvious to stakeholders.
- Use as a natural entry point for learning about the system's architecture.
- Ensure everyone is aware of and agrees with the general system scope.

Description

In a typical context diagram, the system you are developing goes in the center. Draw the various people, software systems, and hardware your system will use or interact with around the system whose context you are describing. Arrows are used to show the relationship among these various elements to describe the overall circumstances in which the system you're designing lives.

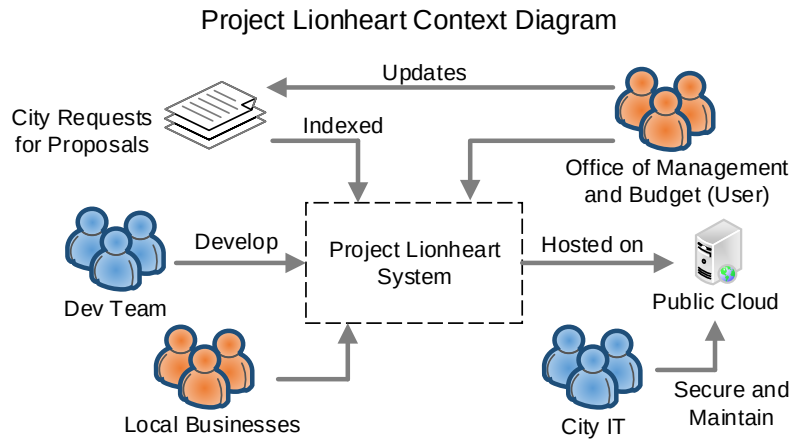
Context diagrams can take on many forms and need not be only a box-and-line diagram. Any graphical depiction that can show where the system fits in the world can work, including drawings, storyboards, cartoon strips, and photographs. Some teams have even experimented with using video and animation to describe a system's context.

Guidelines and Hints

- It's OK to use informal notations. The most important thing is to communicate effectively.
- Show people and systems relevant to the system you are designing.
- Label arrows to tell how two things are related.
- Include a legend to describe the notations in the diagram.

Example

Here is a possible context diagram for Project Lionheart:



Activity 23

Greatest Hits Reading List

As a software system grows, so does the documentation that describes its architecture. A *greatest hits reading list* will help stakeholders navigate the morass of design artifacts so they can find relevant information. Creating a curated reading list provides new stakeholders with a starting point for learning about the architecture.

Benefits

- Highlight the most important design artifacts.
- Provide context for design artifacts within the scope of the whole system.
- Unify disparate, lightweight design artifacts to create a coherent, whole description.

Description

The greatest hits reading list is often a simple link page on the team's wiki. Each link should include the following information:

Title A brief, descriptive title for the artifact. Most artifacts already come with a title.

Overview Briefly explain why this artifact is important or interesting. What should the stakeholder take away from the artifact? It may also be useful to mention when and why the artifact was created.

Caveats In some cases the artifact may be incomplete or outdated. Mention any circumstances the stakeholder should know about when referring to the artifact.

Guidelines and Hints

- Organize the list around stakeholder concerns. Artifacts that address the same concern should be grouped together. Add a heading for the group.
- The same artifact can be used to address different concerns. Use the overview and caveats to help stakeholders navigate an artifact from varying perspectives.

- Take advantage of design artifacts from third-party sources. For example, if a pattern you are using is explained in a framework's documentation or a blog post on the web, use it instead of creating your own document.
- Include links to reference material that defines important concepts in the architecture as well as design artifacts.

Example

Here is an excerpt of a Greatest Hits Reading List from one team's code repository:

WIRE/FIRE/PIRE/TIRE Project: Greatest Hits Reading List

- [Context Diagram](#) - Get a feel for the lay of the land
- [Inception Deck](#) - Created in the first weeks of the project. Much of what's here has changed but it tells why we're building this system.
- [Original system use cases](#) - Largely abandoned but still useful context, skim only.
- [ASR Workbook](#) - Mostly up to date. The top quality attributes still apply.
- [Checkpoint #2 Presentation](#) - Created in March. Includes the most recent architecture diagrams that were shared with all stakeholders.
- [Search and Train sequence diagrams](#) - Shows how different components interact during specific use cases. Useful for availability analysis.
- [Layers Overview](#) - Shows how the code is organized

Activity 24

Inception Deck

Answer ten important questions at the start of a new project to avoid common failures and align stakeholders. The inception deck most often takes the form of a slide deck or lightweight text document and is usually created early in a project's life, during the *inception* phase. The version of the activity outlined here was described by Jonathan Rasmusson in [*The Agile Samurai: How Agile Masters Deliver Great Software* \[Ras10\]](#).

Benefits

- Put important information in the open.
- Share easily with all stakeholders.
- Ensure all stakeholders have a common understanding of important system concerns.
- Discuss important information that should be covered at the start of a new project.

Description

Filling in the inception deck can take as little as 20 minutes when you have the required information handy. Finding the information needed for the deck could take days or weeks. The inception deck is highly customizable. Modify the questions presented here so that they work for your particular situation.

To create an inception deck, answer these questions and record the answers in a slide deck, markdown file, or another format that can be easily shared with stakeholders.

1. Why are we here?

Simply and clearly describe the problem you are going to solve.

2. What's the vision?

Concisely describe how the proposed software system will solve the problem shared by answering the first question. Rasmusson recommends creating an elevator pitch for this slide. There are many resources on this topic available on the web.

3. What's the value?

List the business goals for the project. See [*Discover the Business Goals, on page 43*](#) for ideas and hints.

4. What's in scope?

List the highest-priority functional requirements that are known to currently be in scope. Usually these will be the *must haves*. Also list features that are definitely out of scope as well as functionality with potential architectural significance whose scope is still to be determined.

5. Who are the key stakeholders?

List the key stakeholders and their primary concerns.

6. What does the basic solution look like?

Share a sketch of the notional architecture. This can be an informal diagram such as [*Let Ideas Breathe with a Cartoon, on page 134*](#).

7. What are the key risks? (Why might this project fail?)

List the current top risks in the project. Review what makes a good risk statement in [*Identify Conditions and Consequences, on page 33*](#).

8. How much work? What are the costs?

Using what you know about the scope and notional architecture, estimate the approximate effort and costs to complete the known work. List any assumptions you make about the team size and skill sets.

9. What are the expectations for trade-offs?

Have a frank discussion about key trade-offs before difficult decisions need to be made. Talk about the Big Four: scope, cost, schedule, and quality. Also discuss any interesting or high-priority quality attributes, especially if they may be in tension. Use [*Activity 1, Choose One Thing, on page 192*](#)

10. When will it be ready?

Provide stakeholders with an idea for how long it takes to deliver the software. This estimate is your opportunity to start a conversation about key milestones. Create a draft time line or project schedule for the known work. The schedule is not expected to be perfect and should change as the project evolves.

Once the inception deck is completed, review it with stakeholders and make adjustments based on feedback.

Guidelines and Hints

- Use the ten questions as a checklist for kicking off a software project.
- Periodically review the inception deck as a reminder for what's important in the project.
- Slides are not required! The important thing is to answer the questions. Markdown also works well for the inception deck.
- The effort that goes into creating the Inception Deck should be commiserate with size and cost of the project. For example, don't spend a week creating an inception deck for a two-week project. At the same time, a week might not be enough time to complete an inception deck for a huge, multiteam project.

Example

Jonathan Rasmusson has shared an excellent example on his website.⁷

7. <https://agilewarrior.wordpress.com/2010/11/06/the-agile-inception-deck/>

Activity 25

Modular Decomposition Diagram

Show how the architecture is composed of varying abstractions that come together to create a coherent whole. A *modular decomposition diagram* is a simple tree diagram that shows how varying granularities of abstraction are related to one another. The word decomposition in this context means to break into smaller pieces, not to rot.

Modular decomposition is a general technique that can be used in many circumstances—from code package organization to organization charts to work breakdown structures used in project planning.

Benefits

- Uniquely name concepts at different granularities of abstraction.
- Map refinements in the architecture.
- Use to analyze organizational alignment with the system's composition.
- Reduce complexity without losing traceability to related elements.
- Promote system thinking within the architecture.

Description

Modular decomposition diagrams are nearly always drawn as a tree. The root node of the tree is the *system*. Each level of the tree breaks down (decomposes) a specific module to show finer-grained details. In a large system, the bottom-most leaf nodes might represent a module implemented by a single team. In a smaller system, the bottommost leaf nodes might represent a specific package or class in the architecture.

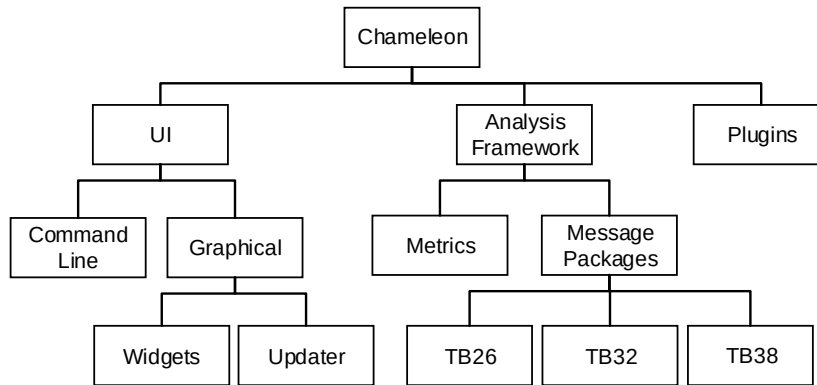
Each level of the tree is an opportunity to group architectural concepts and show how they are related to ideas that are both bigger picture and also more detailed.

Guidelines and Hints

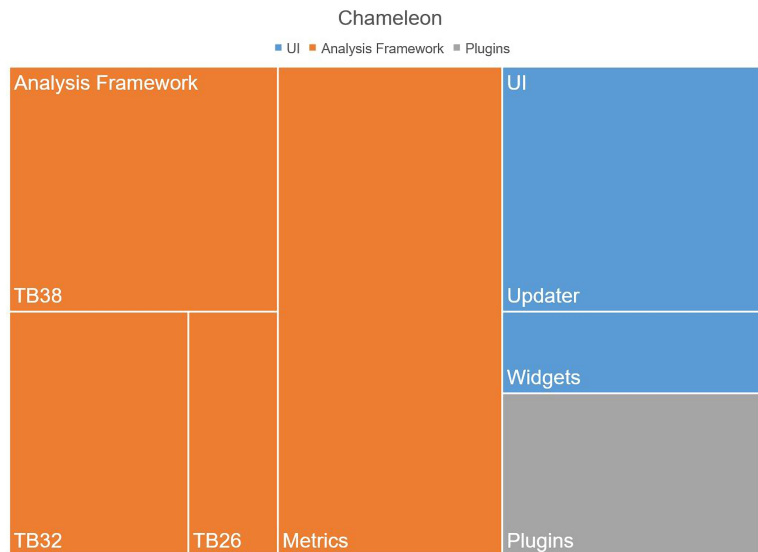
- Use the diagram to help reason about quality attributes such as agility, maintainability, time-to-market, costs, buildability, and deployability.
- Use software to make drawing a tree simpler.

- Break large diagrams into smaller ones to make them easier to understand. Be careful not to lose the context between the diagrams.
- Leaf nodes should not be connected with other leaf nodes except by way of their parent.

Example



Here is the same decomposition drawn as a tree map with some additional information. In this map, size represents the amount of relative technical debt in the module.



Activity 26

Paths Not Taken

Create a list of the architectural decisions you discarded with a brief note explaining why you ultimately rejected each decision. Recording the decisions you discarded, the paths not taken, provides context and rationale for a design decision.

Benefits

- Help downstream designers replay the thought process that went into the current design.
- Head off *did you consider...?* discussions with stakeholders.
- Provide an additional layer of rationale for design decisions.

Description

List design decisions you considered but rejected along with the reason for why the decision was not selected. The list can be stored in plain text or other easily accessible format.

Guidelines and Hints

- Focus on a single view or design decision. The list should not attempt to encompass all design decisions.
- Keep it brief. Include only enough detail so that stakeholders can understand the decision and why it was rejected.
- Combine with other methods to create a more complete description of the architecture. You can list the paths not taken alongside [Architecture Decision Records on page 260](#) and [architecture haikus on page 263](#).

Example

Here is an example of paths not taken for a hybrid cloud project in which an on-premise, shrink-wrapped software system was required to integrate with a rapidly evolving cloud platform on a continuous delivery schedule. The shrink-wrapped software was released quarterly. The scope of the decision centered on the integration between the two platforms.

Path Not Taken	Why the Path Was Rejected
Create a cloud-based “services adapter” to buffer against changes in third-party services	Heavy maintenance costs, benefits in features and quality attributes not required for MVP release, costs outweigh benefits
Release adapter as open source, have customers load it themselves	Extra steps to deployment inhibits adoption, concern that unmodified defaults could introduces security risks, concerns about training customers and consultants, does not reduce maintenance costs
Offer a client-side library	The client library will likely always be out of date (services ship continuously, shrink-wrap software ships once per quarter). Customers must learn both cloud services and how to use the client library. Documentation costs are highest. There is high risk the software will not be ready by the deadline.
No new support for web services integration in the client software	Does not improve the user experience, customers require guidance with emerging patterns and paradigms

The final decision was to release sample code for high-priority use cases in the documentation. The team was not responsible for maintaining the sample code. Customers were on their own if they chose to use or extend the sample code. Stakeholders felt this struck the right balance between costs and value. This minimally viable release allowed the product manager to collect data on the usefulness of the integration before investing further into the architecture.

Activity 27

Prototype to Learn or Decide

Develop or use software so you can test a hypothesis, learn information needed to make a design decision, prove a quality attribute, or gain experience. Sometimes the only way to learn what you need to know is to do it yourself. This adage is especially true of technology and frameworks.

We *prototype to learn* when we need to figure out how to do something or how something works. We *prototype to decide* when we need to gather information that will help us choose between multiple options.

Building a prototype to decide is like running an experiment. The technology or pattern under investigation is hypothesized to solve a specific design problem. The purpose of the prototype is to test the hypothesis.

Benefits

- Gather information through firsthand experience.
- Generate data to use in decision making.
- Allow stakeholders to experience a part of the system.
- Learn how something works quickly and inexpensively.

Description

The difference between a useless prototype and overdoing it is a razor-thin line. To increase your chances of success when prototyping, you need a plan. Let's look at what is involved in creating a prototyping plan.

1. Define the learning objectives and scope for the prototype. What questions will this prototype help you answer?
2. Decide on the prototype's budget and establish a delivery time line. When will you pull the plug on the prototype? How much are you willing to spend to meet the learning objectives? Limit costs and time as much as possible.
3. Decide how the outcomes will be delivered. Who is the audience for the prototype and how will you share it? For example, will you share a demo, whitepaper, presentation, or something else?

The goal is always to implement the prototype as quickly and cheaply as possible. As the prototype comes together, review the implementation relative to the plan. Once you've achieved the objectives, the prototype is complete and it's time to share the outcomes.

After the prototype is complete, perform the minimum clean-up necessary so it can be used again if required. Archive code and instructions for future reference.

Guidelines and Hints

- Look for ways to meet your learning objectives without writing software.
- Decide up front whether the prototype is evolutionary or throw-away.
- Keep tabs on the prototype implementers. Prototyping often requires that you trade quality for speed of delivery and completeness. This is a challenge for many developers who are proud of their craft.
- Time-box the prototype aggressively, but allow sufficient time to complete the work.
- Sketch out a high-level design for the prototype with the implementers before starting development.

Example

Here's an example of prototyping to learn. The team needed to understand the performance limitations of specific Apache Solr APIs. To learn this, a single developer over the course of one week developed a simple test driver using Apache JMeter,⁸ ran several load tests, collected data, and wrote a two-page report summarizing the findings. Using Apache JMeter let the team quickly gather data with minimal effort.

Here's an example of prototyping to choose. The team needed to select a server-side web framework. To help them decide, the team implemented a simple blog application using the top two frameworks. The activity was time-boxed to two days. At the end of the second day, the team discussed pros and cons of each framework based on their experiences and chose a framework.

8. <http://jmeter.apache.org/>

Activity 28

Sequence Diagram

Dynamic structures are difficult to appreciate on paper. Use a *sequence diagram* to show how control or data moves through the system at runtime. Model the flow of logic, data, or control among components.

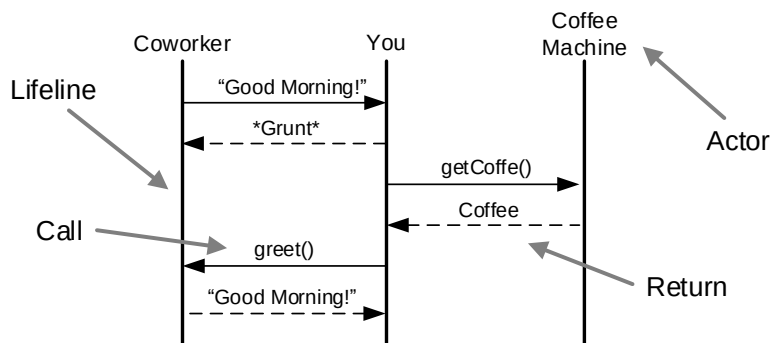
Benefits

- Simple and flexible notation
- Both graphical and text notations exist
- Useful for communication and reasoning
- Ample tool support, though tools are not required

Description

1. Choose a scenario to diagram. Use this as the title of the diagram.
2. List components involved in the scenario horizontally across the top of the page. These are the *participants* in the diagram. Draw a straight, vertical *life line* under each participant. Participants are usually listed from left to right starting with the participant that initiates the scenario.
3. Draw arrows from one participant's life line to another to indicate communication between those components. Label the line to describe the message.
4. Time goes down the y-axis in the diagram. Since the next message happens after the first, it should be further down the y-axis.

A Simple Sequence Diagram

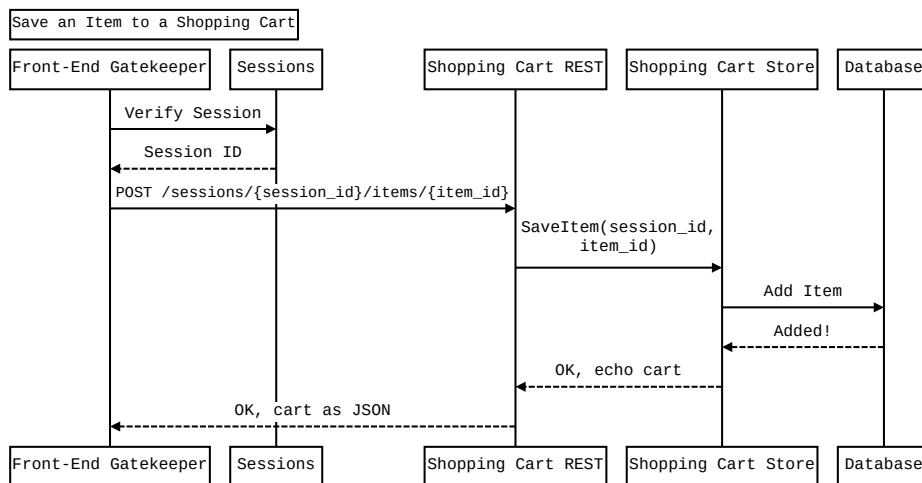


Guidelines and Hints

- Reason about distributed systems, microservices, object communication, and other dynamic structures.
- Informal notations are fine as long as you are consistent.
- A closed arrow with a solid line indicates a *synchronous request message*.
- An open arrow with a solid line indicates an *asynchronous request message*.
- An arrow with a dotted line indicates a *response message*.
- Use a tool that renders text-based notations so you can store the diagrams with your code.

Example

Here is a sequence diagram for a set of microservices responsible for saving an item to a shopping cart, generated using js-sequence-diagrams.⁹



9. <https://github.com/bramp/js-sequence-diagrams>

This is the text notation used to generate the diagram:

```

Title: Save an Item to a Shopping Cart
Front End Gatekeeper -> Sessions: Verify Session
Sessions --> Front End Gatekeeper: Session ID

Front End Gatekeeper -> Shopping Cart REST:
    POST /sessions/{session_id}/items/{item_id}
Shopping Cart REST -> Shopping Cart Store:
    SaveItem(session_id,\n item_id)
Shopping Cart Store -> Database: Add Item

Database --> Shopping Cart Store: Added!
Shopping Cart Store --> Shopping Cart REST: OK, echo cart
Shopping Cart REST --> Front End Gatekeeper: OK, cart as JSON
  
```

With the sequence diagram drawn, it's easy to see a potential flaw in the design. The first step of verifying the session creates an opportunity to validate a session that could be closed—for example, if the user checks out before the item is added. This API call by the *Gatekeeper* is redundant at best. More likely the diagram has exposed an inappropriate assumption about the state of the user's session when saving an item in the *Shopping Cart Store* service.

Activity 29

System Metaphor

Tell a simple story that demonstrates how the system influences specific quality attributes. The system metaphor was introduced by Kent Beck in [Extreme Programming Explained: Embrace Change \[Bec00\]](#) as a way to create a common vision and shared vocabulary for the architecture.

Benefits

- Lightweight description technique perfect for co-located teams to use during times of fast architectural evolution
- Can be combined with other description methods
- Cheap to create, easy to change

Description

In [Making Metaphors That Matter \[KV11\]](#), Michail Velichansky and I summarize concrete guidance for creating useful system metaphors. Good system metaphors have the following attributes:

- Represent a single view of the system.
- Deal with only one type of structure.
- Provide clear guidance concerning design decisions.
- Shed light on system properties.
- Draw on a shared experience.
- Corollary: Even a good metaphor still requires explanation.

Every system metaphor comes with an information payload—the discussions and diagrams that went into creating the metaphor. The metaphor becomes a reference point to this other information and is meant to help team members recall these important details.

Guidelines and Hints

- Tell a memorable story and have fun.
- Be specific and focus on what makes your system unique. Every software system ever made is *like a city*.

- If a common reference point does not exist, then create a shared experience.
- Pop culture and food are common points of reference for many metaphors.
- Common architecture patterns serve the same purpose as system metaphors and can be used in the same way.

Example

In Project Lionheart we plan to build a simple data crawler, which pulls data from the city's *contracts* database, normalizes the data, and pushes it into a search index. To do this efficiently, we'll need a multithreaded crawler. But there's a catch. If the crawler is too aggressive, we might crash the database and disrupt city services. If the crawler is too slow, we might not index the data fast enough to satisfy reliability quality attributes.

To help us think through and share the design, we created the following system metaphor.

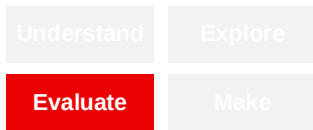
The musical *Newsies*¹⁰ tells the story of the New York City Newsboy Strike of 1899. In the movie (and history), the newsboys (newsies) purchase newspapers from a paper distribution center every morning and then resell the newspapers to the people of New York. Our crawler threads are just like the newsies. Each thread visits a *distribution center*, requests some rows to fetch from the database, and receives a block of rows to fetch and index. Any rows not fetched are forgotten. We can always sell more *papes*—pick up missing rows—tomorrow.

Let's walk through our checklist to see if this is a good metaphor:

- Represent a single view. In this metaphor, we're considering a single view of a single component, specifically the threading model in the crawler. Check.
- Deal with only one type of structure. We're only dealing with C&C structures, so there are no mixed models in play. Check.
- Give clear guidance concerning design decisions. *Newsies* sieze the day and also fetch data to index while the *distribution center* provides a set of rows from the database. Only one *newsie* can have a database row at a time and the *newsies* are fully responsible for the rows once received from the distribution center. Leftover *papers*—database rows not crawled—are skipped. Check.

10. <http://www.imdb.com/title/tt0104990/>

- Shed light on system properties. This metaphor is trying to describe how we'll address performance. We can control our aggressiveness by controlling the number of *newsies* working at the same time. Check.
- Draw on shared experiences. The movie is a piece of 1990s cinematic gold starring a young Christian Bale. If teammates haven't seen it, create a shared experience by ordering pizzas and having a movie night, or at least watching YouTube videos. Check and team building bonus!



CHAPTER 17

Activities to Evaluate Design Options

In the evaluate mindset, we critically examine design decisions to determine how well they meet our needs. Our designs don't need to be perfect, but they do need to be good enough. Our goal is to make sure the architecture *satisfices*, that it is satisfactory and sufficient. When we've found a satisficing solution, then we say it has good fit.

During an evaluation we'll learn all the ways our architecture is not satisfactory or sufficient. We might learn we don't understand some nuance about the problem. Or perhaps a design idea that seemed good will turn out to have unacceptable trade-offs, miss an important constraint, or introduce too much risk. These things are better to know early, before it becomes difficult to change a potentially costly decision.

After an evaluation we should have enough information to decide which design mindset to embrace next. We'll always embrace the evaluate mindset during the *check* step in the do-make-check cycle, but evaluation can be the main attraction during the *do* step too.

Evaluation is a continuous activity. Waiting until the end of a design phase to do an evaluation is too late. We should evaluate our work every step of the way. Once we deem some part of the architecture to be good enough then that part of the system's design is ready to be refined further by focusing on finer-grained details. Everything in the architecture need not be *ready* before starting to build something.

The activities in this chapter help teams look deeply at different facets of the architecture and glean information needed to take action. Use them when you need to check your understanding, choose a design option, or help you figure out what to do next.

Activity 30

Architecture Briefing

This brief (no pun intended) presentation is used to bring stakeholders up to speed about some part of the architecture. By the end of the briefing, participants are prepared to provide meaningful feedback about the architecture.

Architecture briefings are a common practice used by traditional building architects to educate clients and share progress. The same general practice has been used in software development for decades. The idea of using an *architecture briefing* with software has been proposed by many, including Stuart Halloway¹ and Patrick Kua.²

Benefits

- Quickly bring stakeholders up to speed so they can ask questions and point out issues in a design.
- Foster a sense of shared ownership over the architecture.
- Enable more stakeholders to provide feedback, ensuring a diversity of perspectives critically evaluate the design.
- Promote accountability in architectural decision making.
- Create a platform for teaching and learning architecture design. Teammates will be exposed to other people's approaches to architecture design and have a chance to practice articulating designs concisely.

Activity Timing

Forty-five minutes to one hour. The presentation portion of the briefing should last no more than 30 minutes. Try to leave at least half of the meeting time for audience questions and feedback.

Participants

The architect presents the briefing.

1. <https://github.com/stuarthalloway/presentations/wiki/Architectural-Briefings>
2. Patrick Kua. *Evolutionary Architecture*. SATURN 2016. <http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=454345>

Stakeholders and knowledgeable nonstakeholders attend the briefing. The briefing should be open to a wide audience, including people with no prior knowledge of the architecture under discussion.

Preparation and Materials

- Briefing presentation. This will usually be slides but can also be a white-board talk. As a rule of thumb, preparing for the briefing should require no more than about twice the briefing length. For example, preparing for a 30-minute presentation should take about an hour.
- Audience members should bring supplies to take notes.

Steps

1. Welcome the audience, introduce the architect, and share the ground rules for the briefing. Here are some sample ground rules you can use:

Audience's job: Question everything.

Please hold questions and comments until the end.

Pay attention. Take notes.

Think about: What is missing? How does this compare to your experience? Do you agree with the decisions? Do you understand why a decision was made?

Be respectful; remember, your briefing is next!

2. The architect presents the briefing. The presentation ends when time expires or the architect is done, whichever comes first.
3. Open the floor for comments and questions. Team members may optionally join the architect to field questions.
4. Conclude the briefing by thanking the audience and architect. The briefing ends when there are no more questions or the time has expired.

Guidelines and Hints

- Host briefings at the same time and place at a regular interval.
- Publish slides after the briefing and record it if possible.
- The team presenting the briefing should appoint a note taker during the questions portion of the briefing.
- Audience questions should be tough, but constructive.

Example

Here is one possible outline for architecture briefings. The specific outline may vary depending on the type of systems you build.

Elevator pitch: what overall business problem are you solving?

Overview and context

Top quality attributes

Relevant views

Key design decisions with rationale

Alternatives considered

Current status: quality, work remaining, next steps

Costs

Top risks and other concerns

Future plans

Halloway included an example briefing and an alternative outline in a talk available in his GitHub repository.³

3. <https://github.com/stuarthalloway/presentations/wiki/Architectural-Briefings>

Activity 31

Code Review

A form of peer review in which code is incrementally inspected with an eye toward architectural concerns as the code is developed. Code review is a fantastic practice that every team should do anyway. Extending code reviews to include architectural concerns makes them even more powerful.

Incrementally inspecting code as the architecture manifests helps fight architectural rot by keeping tabs on the system's evolution relative to the planned design. Reviews are also a great time to identify design inflection points that emerge during development. Such inflection points may require further analysis.

Every review presents a potential coaching opportunity. Watch out for synchronization mismatches in mental models so you can fix them before real problems arise. Coach teammates on architectural principles as well as the specific architecture you're developing together.

Benefits

- Keep architecture design at the forefront of every developer's mind.
- Allow finer-grained details to emerge without losing a connection with coarser-grained architectural concerns.
- Manage emergent details that can cause problems in the architecture.
- Influence the detailed design as required.
- Create teachable moments to grow the team's architecture design competence.

Activity Timing

Ongoing for the life of a project. An initial code review might take as little as 10 minutes with more time needed to resolve any identified issues.

Participants

The author submits an artifact, in this case code, for review. The reviewer inspects the artifact and provides feedback. Many teams encourage multiple reviewers to participate.

Preparation and Materials

- A prepared code artifact, patch, or pull request.

Steps

1. Skim the change set to get a feel for the overall scope of changes.
2. Perform the code review as you normally would, focusing on detailed design, style issues, and defects.
3. Once the first pass review is complete, reflect on potential architectural implications of the change set. See the [sample checklist on page 291](#) to get an idea of what you might look for during this part of the review.
4. Add comments related to your architectural concerns. If there are potential gaps in understanding, reference relevant resources.
5. After sharing your review with the submitter, follow up with that person directly. Architectural issues usually require more discussion.
6. Reflect on the results of the review. Were there issues that could have been avoided with more education or documentation? Is there an implied design decisions that should be explicitly stated? Add design tasks to the backlog to address ideas you think will bear fruit, such as improving a document or hosting an information session with the team.

Depending on the tool or exact situation for your code review, these steps may need to be adapted.

Guidelines and Hints

- Use code review software that integrates with your version control and build system.
- Reviews are often small. Watch out for thematic shifts in how the code evolves over time.
- Escalate the style of peer review to resolve issues quickly. For example, shift to pair programming or host a [whiteboard jam on page 255](#) when an issue arises due to a lack of understanding.
- It's OK for the architecture to change as the system emerges. The primary goals of code review from the architect's perspective is to increase awareness of design decisions, monitor the implementation of the architecture, and guide change over time.

- Code review is not a replacement for design evaluations. Use reviews to monitor architectural drift and learn how you can better serve your team as an architect.

Example

Code review checklists improve consistency and show teammates what to look for during a review. In addition to looking at detailed design concerns, here are some architectural ideas you should look for when reviewing code:

- **Correctness**—Are the changes consistent with the established patterns in the architecture? Are there pattern violations? Is there an opportunity to use an architecture pattern or refactor the code so that an intended pattern becomes more obvious?
- **Consistency**—Look at the naming. Do the concepts at play make sense? Do any names surprise you? Are you able to form a mental model in your head of where the changes fit? How well does this jive with your expectations of what these changes would entail?
- **Testability**—Are there clean unit tests included with the changes? Can the tests be run with every build? Is there an opportunity for the tests to be flaky or inconsistent? Are common patterns such as *inversion of control* used appropriately and correctly?
- **Modifiability/maintainability**—Are there hard-coded constants or values that should be injected via configuration? What assumptions are baked into the code under review about what will change in the system? Can the code be made more flexible? Were any new dependencies introduced? Why were they introduced? Was it right to include them?
- **Reliability**—Are exceptions handled consistently? Are there opportunities for errors to propagate in unexpected or unhandled ways? Does the system attempt to retry when appropriate? Does the system fail fast when no recovery action can be attempted? How is error prevention (including from human mistakes) built into the design?
- **Scalability**—Does the code introduce potential for rampant memory use? Are the algorithms at least nominally efficient? Are thread-safe data structures used when appropriate?

Activity 32**Decision Matrix**

A decision matrix is a visual comparison of how various alternatives stack up against one another. Use a decision matrix to qualify design alternatives so a decision can be made. A decision matrix can also be used in documentation as a part of the design's rationale.

Benefits

- Use to compare a variety of decisions such as patterns, technologies, or frameworks.
- Visualize relative strengths and weaknesses among decisions.
- Focus attention on essential factors when comparing and contrasting alternatives.
- Facilitate open discussion about trade-offs among alternatives.

Activity Timing

Varies, depending on the number of alternatives and evaluation factors.

Participants

The architect is responsible for ensuring the matrix is filled in accurately. Stakeholders validate the evaluation factors.

Preparation and Materials

- Identify a list of architecturally significant requirements, especially quality attribute scenarios to be used as the properties for comparison.
- Before starting the analysis, identify at least two design alternatives for comparison.

Steps

1. Identify evaluation factors. Collaborate with stakeholders to agree on the factors used to compare and contrast alternatives.
2. Establish a rubric. Collaborate with stakeholders to decide how design alternatives will be scored. For guidance on defining a rubric, see [Define a Design Rubric, on page 162](#).
3. Do the analysis and fill in the matrix.
4. Share the matrix with stakeholders. Verify the analysis and discuss your recommended decision.

Guidelines and Hints

- Use qualitative comparisons unless you performed quantitative analysis. For example, performance or availability can only be quantified if you ran tests.
- Consider no more than seven factors in the same matrix.
- Compare up to five design options in the same matrix. Use multiple matrices with a larger number of options.
- Take good notes when filling in the matrix. The analysis is as important as the results and can provide design rationale for decisions.

Example

Here is a sample decision matrix. Additional examples are shown in [Create a Decision Matrix](#), on page 71.

Project Lionheart Decision Matrix

	3-Tier	Publish - Subscribe	Service Oriented
Availability (Database unavailable)	+	○	+
Availability (Uptime requirements)	○	○	○
Performance (5-second response time)	○	-	+
Security	○	-	○
Scalability (5% annual growth)	○	○	+
Maintainability (Team knowledge)	+	-	○
Buildability (Implementation risks)	++	-	--

Legend

Strongly Promotes	++	Strongly Inhibits	--	Neutral	○
Promotes	+	Inhibits	-		